SMART INDIA HACKATHON · SIH 26262

AICTE Approval & Compliance Orchestration Platform

Teammate Handover, Live Demonstration & Viva Presentation Guide

AICTE Approval & Compliance Orchestration Platform (SIH 26262)

Teammate Handover, Evaluation & Viva Presentation Guide

Project: AICTE Approval & Compliance Orchestration Platform

Problem Statement: SIH 26262

Target Audience: Team Members, Presentation Speakers, Demo Leads, and Viva Presenters

Target Working Directory: `/home/shreyash/Projects/SIH2`

Shareable ZIP Deliverable: `/home/shreyash/Projects/SIH2/SIH26262_Complete_Project.zip`

Document Generation Date: October 23, 2026

Table of Contents

1. Handover Quick Reference
2. Fast-Start 3-Terminal Execution Guide
3. DBeaver & Database GUI Setup
4. 30-Second Elevator Pitch (English & Hinglish)
5. 1-Minute Architecture Pitch (English & Hinglish)
6. Step-by-Step Live Demo Script for Evaluators
7. What Is Live vs. What Is Demo (Never Confuse These!)
8. 18 Tough Judge Counter-Questions & Exact Model Answers
9. Troubleshooting & Emergency Fixes

1. Handover Quick Reference

Asset	Location	Purpose
Complete Project ZIP	`/home/shreyash/Projects/SIH2/SIH26262_Complete_Project.zip`	1,010 KB clean archive to share with teammates/evaluators
Technical Architecture Report	`SIH26262_Complete_Technical_Report.md`	Comprehensive 86 KB technical reference manual
Technical Architecture PDF	`SIH26262_Complete_Technical_Report.pdf`	Formatted handover PDF for submission & printing
API Documentation	`SIH26262_API_Documentation.md`	Complete REST API endpoint reference & cURL cookbook
Testing Guide PDF	`docs/SIH26262_Backend_Connected_UI_Testing_Guide.pdf`	Step-by-step UI testing guide with PostgreSQL verification
Backend Code	`src/main/java/in/gov/aicte/platform/...`	Spring Boot 3.4.3 backend state machine engine
Frontend Code	`frontend/src/...`	React 19.3.1 + Vite 8.3.5 client application
Flyway SQL Migrations	`src/main/resources/db/migration/`	Versioned schema migrations `V1` and `V2`

2. Fast-Start 3-Terminal Execution Guide

If a teammate has just unzipped `SIH26262_Complete_Project.zip`, open **3 separate terminal tabs** from the project root:

BASH

```
unzip SIH26262_Complete_Project.zip
cd SIH2
```

Terminal 1: Start PostgreSQL Database

BASH

```
docker compose up -d

# Check if ready
pg_isready -h localhost -p 5432
```

Note: If Docker is not available, ensure a native PostgreSQL 16+ instance is running with database `aicte_platform_db` on port 5432, user `aicte_app_user`, password `aicte_123`.

Terminal 2: Start Spring Boot Backend

BASH

```
./mvnw spring-boot:run
```

Wait for output:

`Started AictePlatformApplication in X.XXX seconds (process running for Y.YYY)`

Test health:

BASH

```
curl http://localhost:8080/api/v1/health
# Returns: {"status":"UP","application":"aicte-compliance-platform",...}
```

Terminal 3: Start React 19 Frontend

BASH

```
cd frontend
npm ci
npm run dev
```

Open your browser at: **`http://localhost:5173`**

---

3. DBeaver & Database GUI Setup

To show live database records directly to evaluators:

1. Open **DBeaver** (or pgAdmin / VS Code Database Client).
2. Create a new **PostgreSQL** connection:

- **Host**: `localhost`
- **Port**: `5432`
- **Database**: `aicte_platform_db`
- **Username**: `aicte_app_user`
- **Password**: `aicte_123`

1. Click **Test Connection** -> **Finish**.
2. Navigate to `Databases` -> `aicte_platform_db` -> `Schemas` -> `public` -> `Tables`.
3. You will see 5 tables:

- `applications`
- `institutions`
- `workflow_transitions`
- `audit_events`
- `flyway_schema_history`

---

4. 30-Second Elevator Pitch (English & Hinglish)

English (Standard Academic Presentation)

> *"The AICTE Approval & Compliance Orchestration Platform replaces fragmented, paper-heavy technical education approvals with a state-machine-governed regulatory backbone. Built on Java 21, Spring Boot 3.4.3, PostgreSQL 18, and React 19, the platform enforces strict server-side state transitions. When a Scrutiny Officer detects deficiencies, returns an application, and the college submits rectifications, every transition is atomically recorded in PostgreSQL with an immutable audit log and optimistic locking protection."*

Hinglish (Conversational Pitch for Quick Judges / Jury)

> *"Hamara platform AICTE ke approval process ko digital workflow me automate karta hai. Aaj ke time pe agar college me koi deficiency nikalti hai—jaise fire NOC expired ya faculty roster incomplete—toh sab kuch emails aur physical letters pe chalta hai jisse approval mahino atka rehta hai.*

> *Humne ek centralized state-machine engine banaya hai. Agar Scrutiny Officer application return karega, toh system deficiency save karke ownership college ko assign karta hai. College jab rectification submit karega, tabhi status 'In progress' hoga aur file wapas officer ke pass aayegi. Ye saara data client-side nahi, balki Spring Boot backend ke through PostgreSQL me transaction-safe tarike se save hota hai, aur har action ka immutable audit log banta hai."*

---

5. 1-Minute Architecture Pitch (English & Hinglish)

English Architecture Walkthrough

> *"Our architecture enforces that **the backend owns business logic, state machines, and regulatory validation**. The React 19 frontend is a client interface that communicates with our Spring Boot backend via RESTful APIs. When a user executes a workflow action, it passes through Spring Security's stateless filter chain where our development Bearer token bridge verifies the caller's server-resolved profile.

> The request hits our `@Transactional` `WorkflowService`. Here, three critical operations occur in a single database transaction: first, the `applications` entity is updated with an optimistic locking `@Version` increment; second, an immutable transition record is inserted into `workflow_transitions`; and third, a regulatory audit entry is written to `audit_events` with Jackson-serialized metadata. Database schema integrity is managed via Flyway migrations, ensuring zero manual schema drift."*

Hinglish Architecture Walkthrough

> *"Architecture wise, humne backend-first design follow kiya hai. Client side pe React 19 SPA hai aur backend pe Spring Boot 3.4.3 REST API. Jab koi officer action submit karta hai, request Spring Security ke stateless filter chain me aati hai jahan dev-bearer filter user ka verified profile inject karta hai.

> Iske baad `WorkflowService` Spring ke `@Transactional` boundary ke andar 3 cheeze atomically execute karta hai: pehla, application ka stage aur status update hota hai with version increment taaki race conditions na hon; doosra, transition history insert hoti hai; aur teesra, Jackson-serialized audit event create hota hai. Agar koi bhi failure aayi toh complete rollback ho jata hai. Database schema Flyway versioned migrations (`V1`, `V2`) se automatically govern hota hai."*

---

6. Step-by-Step Live Demo Script for Evaluators

Follow this exact 8-step script during your viva demo:

Step 1: Open the Frontend and Introduce the Role Switcher

- Open `http://localhost:5173`.
- Point out the **Header Role Switcher** (top-right).
- Switch role to **`Scrutiny Officer`** (`B. Singh`).
- Explain: *"When I switch roles, the frontend requests an authentic Bearer token from `POST /api/v1/auth/token`. The backend resolves the user's role on the server—we never trust client-supplied roles in action payloads."*

Step 2: Open the Live Application Dossier

- Navigate to **Applications Explorer** (`/applications`).
- Click on **`APP-2026-1042`** (*ABC Institute of Technology*).
- Point out: *"Notice this application is live from PostgreSQL 18. The current stage is `Scrutiny`, status is `In progress`, and the assigned owner is Scrutiny Officer B. Singh."*

Step 3: Execute "Return for Correction"

- Click the amber button: **"Return for Correction"**.
- Enter Reason:

```
Fire safety NOC expired. Faculty roster missing HoD signature.
```

- Click **Submit Return for Correction**.
- Point out:
- The status immediately updates to **`Returned`**.
- Current stage advances to **`Re-scrutiny`**.
- Current owner changes to **`Institution Representative`**.
- An amber banner displays the cited deficiencies.
- The workflow timeline appends the return event.

Step 4: Switch to Institution Representative & View Objections

- Switch role in the header to **`Institution Representative`** (`Dr. Ramesh`).
- Notice the application dossier reflects the `Returned` status from the college's perspective.
- The button **"Submit Deficiency Rectification"** is now active.

Step 5: Submit Deficiency Rectification

- Click **"Submit Deficiency Rectification"**.
- Enter explanation:

```
Uploaded renewed Fire NOC from Delhi Fire Service (valid till 2029) and countersigned faculty roster.
```

- Click **Submit Rectifications**.
- Point out:
- Status updates to **`In progress`**.
- Ownership transfers back to **`B. Singh (Scrutiny Officer)`**.
- Deficiencies are cleared upon formal resubmission.

Step 6: Switch back to Scrutiny Officer & Complete Scrutiny

- Switch role back to **`Scrutiny Officer`**.
- Review the cleared dossier.
- Click the green button: **"Complete Scrutiny & Forward"**.
- Select target stage: **`EVC scheduling`** (Physical / Hybrid Inspection).

- Click **Confirm Clearance**.
- Point out:
- Stage is now `EVC scheduling`.
- Status is `Due soon`.
- Ownership is assigned to `C. Verma (EVC Coordinator)`.

Step 7: Prove Persistence Directly in PostgreSQL (The Winning Moment!)

- Switch to your terminal or DBeaver and run:

```bash
PGPASSWORD=aicte_123 psql -h localhost -U aicte_app_user -d aicte_platform_db -c "
SELECT application_reference, current_stage, status, owner_name, version FROM applications
SELECT action, from_stage, to_stage, from_status, to_status, actor_name, created_at FROM w
"
```

- Show the evaluator the real rows in PostgreSQL! Point out the `version` incremented from `0` to `3`!

Step 8: Show EVC Inspection Workbench

- Navigate to **EVC Operations** (`/evc`).
- Show the EVC inspection workbench with its 5 canonical tabs (`EVC Queue`, `Scheduled Visits`, `Upcoming Visits`, `Completed Visits`, `Reports Pending`).
- Explain: *"Here our frontend state machine manages expert committee rosters, physical visit dates, and 48-hour report submission clocks."*

---

7. What Is Live vs. What Is Demo (Never Confuse These!)

Feature	Where Data Lives	How to Explain Honestly to Judges
Application List & Filtering	**PostgreSQL 18**	*"Live backend REST API using Spring Data JPA with `@EntityGraph` eager loading to prevent N+1 query overhead."*
New Application Registration	**PostgreSQL 18**	*"Live backend REST API (`POST /api/v1/applications`) with unique constraint validation and auto-refresh."*
Scrutiny State Machine	**PostgreSQL 18**	*"Live backend transactional engine enforcing prerequisites, optimistic locking, and atomic audit logging."*
Audit Log Trail	**PostgreSQL 18**	*"Live backend tables (`workflow_transitions` and `audit_events`) using Jackson serialization."*
EVC Inspection Workbench	**Browser LocalStorage**	*"Client-side prototype state machine backed by localStorage and verified by 6 unit tests. Backend persistence is mapped for Phase 2."*
Compliance Obligations	**Browser LocalStorage**	*"Illustrative compliance dashboard demonstrating affidavit tracking and statutory compliance monitoring."*
Document Registry	**Browser LocalStorage**	*"Prototype document registry simulating version tracking; Phase 2 will integrate MinIO object storage."*
Executive Dashboards	**Hybrid**	*"Application metrics and regional distributions are calculated live from backend APIs; compliance and EVC delay metrics are illustrative demo data."*

8. 18 Tough Judge Counter-Questions & Exact Model Answers

Q1: Why did you choose Spring Boot over Python (Django/FastAPI) or Node.js?

Answer: *"For government regulatory and banking platforms, compile-time type safety, declarative transaction management (`@Transactional`), and mature ORMs with optimistic locking are industry gold standards. Spring Boot guarantees that application state and audit events succeed or fail atomically. However, we acknowledge that Django or NestJS could also achieve similar functional outcomes if architected with equal discipline."*

Q2: Is your authentication production-ready?

Answer: *"No, and we are completely transparent about that. For this prototype, we implemented a server-side Development Bearer Token Bridge that simulates enterprise role resolution without requiring external LDAP/OAuth infrastructure. In a production AICTE rollout, this will be replaced with AICTE Parichay or DigiLocker OIDC OAuth2 SSO."*

Q3: What prevents concurrent officers from overwriting each other's changes?

Answer: *"We implemented JPA Optimistic Locking using the `@Version` column on the `applications` table. If two officers read version 1, and Officer A saves first (incrementing version to 2), Officer B's update will fail*

with an `OptimisticLockingFailureException`, which our `GlobalExceptionHandler` converts to an RFC 7807 HTTP 409 Conflict response."

Q4: How do you handle database migrations between development and production?

Answer: *"We use Flyway Core versioned migrations. All schema definitions (`V1` for core tables, `V2` for workflow extensions and demo seed data) are written in version-controlled SQL files. When the backend starts, Flyway inspects `flyway_schema_history` and applies pending migrations automatically."*

Q5: How did you fix the N+1 select query problem?

Answer: *"In our `ApplicationRepository`, we added `@EntityGraph(attributePaths = {"institution"})` to `findAll` and search queries. This instructs Hibernate to perform a single SQL `LEFT OUTER JOIN` between `applications` and `institutions`, eliminating the N+1 query bottleneck during paginated lookups."*

Q6: Can a college representative approve their own application?

Answer: *"No. The state machine in `WorkflowService` strictly validates actions against the caller's server-resolved role. Only users with the `Scrutiny Officer` role can execute `COMPLETE_SCRUTINY` or `RETURN_FOR_CORRECTION`, while `Institution Representative` can only execute `SUBMIT_CORRECTIONS`."*

Q7: Can scrutiny be cleared if deficiencies are still unresolved?

Answer: *"No. In the frontend, the 'Complete Scrutiny' button is disabled while open deficiencies exist. In the backend, `SUBMIT_CORRECTIONS` requires an explanatory note, and `COMPLETE_SCRUTINY` validates that the application is in an approvable state."*

Q8: What happens if the database crashes while recording an audit event?

Answer: *"Because `executeWorkflowAction` is annotated with `@Transactional`, the application status update, the transition history row, and the audit event row are committed in the same database transaction. If the database crashes, PostgreSQL's write-ahead log automatically rolls back the entire transaction, ensuring zero orphaned or corrupted records."*

Q9: Why do you have two separate tables for transitions and audit events?

Answer: *"They address different architectural concerns. `workflow_transitions` is an explicit state-machine history capturing state deltas (`fromStage` -> `toStage`, `fromStatus` -> `toStatus`). `audit_events` is a generalized, immutable regulatory log recording the actor, target entity, and Jackson-serialized JSON metadata across all modules."*

Q10: How do you store documents? Are they binary blobs in PostgreSQL?

Answer: *"No, storing large PDF binaries in relational databases degrades database performance. In this prototype, documents are referenced as metadata. In our Phase 2 roadmap, document files will be stored in an S3-compatible object storage service (such as MinIO), storing only document metadata and SHA-256 integrity hashes in PostgreSQL."*

Q11: How do you handle malformed JSON requests?

Answer: *"Our `GlobalExceptionHandler` catches `HttpMessageNotReadableException`, `MethodArgumentNotValidException`, and `MethodArgumentTypeMismatchException`, formatting them into standardized RFC 7807 Problem Detail responses with specific error URIs, titles, and timestamped invalid field maps."*

Q12: Is your application Dockerized?

Answer: *"Yes. We provide a `compose.yaml` specification for our PostgreSQL 18 container with persistent storage volumes and health checks. For the backend and frontend, running `./mvnw spring-boot:run` and `npm run dev` enables instant hot-reloading and debugging during development."*

Q13: Can an Institution Representative see another college's applications?

Answer: *"In this prototype, role-level security is enforced (only Institution Reps can submit corrections, only Scrutiny Officers can clear scrutiny), but resource-level multitenant filtering (matching `user.institutionId == app.institutionId`) is planned for Phase 2 using Spring Security `@PreAuthorize` annotations."*

Q14: How did you fix the recent EVC premature completion bug?

Answer: *"We identified that the action button for scheduled visits was literally labeled 'Visit Completed' with green styling, making it look like a completed status pill instead of an action. Clicking it also invoked completion. We created a canonical `evcLifecycle.js` module, renamed the button to imperative 'Mark Completed' with an outline action style, added a server-style guard blocking un-scheduled visits, and added a safe manual correction path to revert accidental completions."*

Q15: How did you fix the `deficienciesList is not defined` bug?

Answer: *"In `ApplicationDetailPage.jsx`, the clearance confirmation handler was referencing an undefined variable `deficienciesList`. We traced the state and corrected the reference to use `app.deficiencies || []` from the loaded application dossier, preserving deficiency validation before clearance."*

Q16: How do you scale this system for 10,000+ colleges across India?

Answer: *"The Spring Boot backend is completely stateless; it can horizontally scale across multiple container instances behind an AWS ALB or Nginx reverse proxy. On the database layer, read replicas can handle high-volume dashboard queries while the primary PostgreSQL node handles transactional writes. Connection pooling is managed by HikariCP."*

Q17: What automated tests have you written?

Answer: *"We have 20 automated backend integration and unit tests passing in Maven (testing health, persistence, CRUD, pagination, error responses, Jackson serialization, and workflow transitions) and 6 automated frontend lifecycle tests passing in Node (testing scheduling invariants, tab filters, and completion guards)."*

Q18: What is your primary technical achievement in this prototype?

Answer: *"Delivering an end-to-end, transactionally coordinated regulatory scrutiny journey where live Spring Boot REST APIs, PostgreSQL Flyway migrations, and a responsive React 19 UI interact seamlessly with zero broken builds, verified audit trails, and zero mock data conflation on core workflows."*

---

9. Troubleshooting & Emergency Fixes

Issue 1: Port 8080 or Port 5432 is already in use

BASH

```
# Find and terminate process on port 8080
sudo lsof -i :8080
sudo kill -9 <PID>

# Find process on port 5432
sudo lsof -i :5432
```

Issue 2: Docker Compose database container is stopped

BASH

```
cd /home/shreyash/Projects/SIH2
docker compose down
docker compose up -d
pg_isready -h localhost -p 5432
```

Issue 3: Re-running backend tests

BASH

```
cd /home/shreyash/Projects/SIH2
./mvnw clean test
```

Issue 4: Re-building frontend production assets

BASH

```
cd /home/shreyash/Projects/SIH2/frontend
npm run build
```

End of SIH 26262 Team Handover Guide.